Manual Tecnico - PromptZal-V3

Daren Uziel Zapeta Molina- 202132253

1. Introducción y Objetivos

1.1. Introducción

El proyecto **PromptZal** consiste en el diseño e implementación de un compilador completo. Su propósito fundamental es recibir código fuente estructurado con la extensión .pz, el cual define directivas globales, agentes de inteligencia artificial, contextos, variables y comandos operativos, para transformarlo y traducirlo de manera automatizada en prompts de sistema optimizados en lenguaje natural.

Para cumplir con este ciclo de vida, el compilador implementa un pipeline robusto y modular dividido en etapas estrictas:

- **Análisis Léxico:** Encargado de escanear la secuencia de caracteres fuente y agruparlos en componentes atómicos válidos (tokens) mediante reglas regulares definidas con JFlex.
- **Análisis Sintáctico:** Procesa los tokens mediante un analizador de descenso recursivo escrito a mano, aplicando recuperación por modo pánico para construir simultáneamente el Árbol de Sintaxis Abstracta (AST).
- **Análisis Semántico:** Recorre el AST de manera integral para validar los ámbitos, la unicidad de agentes y la existencia previa de variables, interactuando directamente con una Tabla de Símbolos centralizada.
- **Traducción y Reportes:** Produce las instrucciones de sistema finales y genera de manera desacoplada reportes diagnósticos detallados en formato HTML (tokens, errores, tabla de símbolos y AST).

1.2. Objetivos del Proyecto

Objetivo General

- Desarrollar un compilador funcional y tolerante a fallos para el lenguaje PromptZal, aplicando los fundamentos teóricos y prácticos de la compilación moderna, lenguajes formales y diseño orientado a objetos.

Objetivos Específicos

1. **Implementar el Análisis Léxico:** Configurar el escáner con JFlex para reconocer palabras reservadas, operadores, directivas y literales, capturando y aislando errores léxicos sin abortar la ejecución.
2. **Construir el Analizador Sintáctico:** Desarrollar un parser descendente recursivo guiado por una gramática libre de contexto factorizada, asegurando la correcta jerarquía del AST y la recuperación ante fallos sintácticos.

3. **Ejecutar la Validación Semántica:** Comprobar la coherencia lógica, control de tipos de ámbito local/global y visibilidad de identificadores mediante un analizador semántico y una estructura de tabla de símbolos.
4. **Generar Reportes Diagnósticos:** Proveer una interfaz de visualización limpia mediante la exportación de resultados a reportes HTML estructurados para facilitar la depuración del código fuente.

2. Definición Formal del Lenguaje PromptZal

El lenguaje PromptZal se compone de un conjunto finito de terminales (tokens) reconocidos mediante el análisis léxico, y una estructura sintáctica definida a través de una Gramática Libre de Contexto en formato BNF (Backus-Naur Form).

2.1. Tabla de Terminales (Tokens)

Para los terminales se usaron los siguientes:

- **DIRECTIVA:** `@modelo`, `@rol`, `@formato`
- **AGENTE:** `AGENTE`
- **CONTEXTO:** `contexto`
- **VARIABLE:** `variable`
- **EJECUTAR:** `EJECUTAR`
- **EXPORTAR:** `EXPORTAR`
- **COMANDO_IA:** `PREGUNTAR`, `GENERAR`, `RESUMIR`, `ANALIZAR`, `TRADUCIR`, `CLASIFICAR`, `EXTRAER`
- **CARGAR:** `CARGAR`
- **CONECTOR:** `SOBRE`, `DESDE`, `EN`, `COMO`
- **FLECHA:** `->`
- **IGUAL:** `=`
- **MAS:** `+`
- **LLAVE_A / LLAVE_C:** `{` / `}`
- **PAR_A / PAR_C:** `(` / `)`
- **COMA:** `,`
- **ID:** Identificadores (nombres de agentes, variables y palabras como unidades)
- **CADENA:** Texto delimitado por comillas dobles (`"..."`)
- **NUMERO:** Valores numéricos enteros y decimales
- **EOF:** Fin de archivo (generado por el lexer al terminar)

2.2. Gramática Libre de Contexto (BNF)

La siguiente gramática (Se uso la recomendada en el enunciado) describe la estructura y jerarquía completa de un programa válido. Los símbolos entre `< >` representan a los no terminales (que se traducen a métodos en el parser), los textos en MAYÚSCULAS representan a los terminales (tokens de la tabla anterior), y el símbolo `ε` indica una producción vacía.

<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> -> DIRECTIVA CADENA <directivas> | ε

<agentes> -> <agente> <mas_agentes>

<mas_agentes> -> <agente> <mas_agentes> | ε

<agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> -> CONTEXTO IGUAL CADENA

<variables> -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos> -> <comando> <mas_comandos>

<mas_comandos> -> <comando> <mas_comandos> | ε

<comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores> -> CONECTOR <expresion> <conectores> | ε

<expresion> -> <termino> <mas_terminos>

<mas_terminos> -> MAS <termino> <mas_terminos> | ε

<termino> -> CADENA | ID | NUMERO <unidad> | <cargar>

<unidad> -> ID | ε

<cargar> -> CARGAR PAR_A CADENA PAR_C

<ejecuciones> -> EJECUTAR ID <mas_ejecuciones>

<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

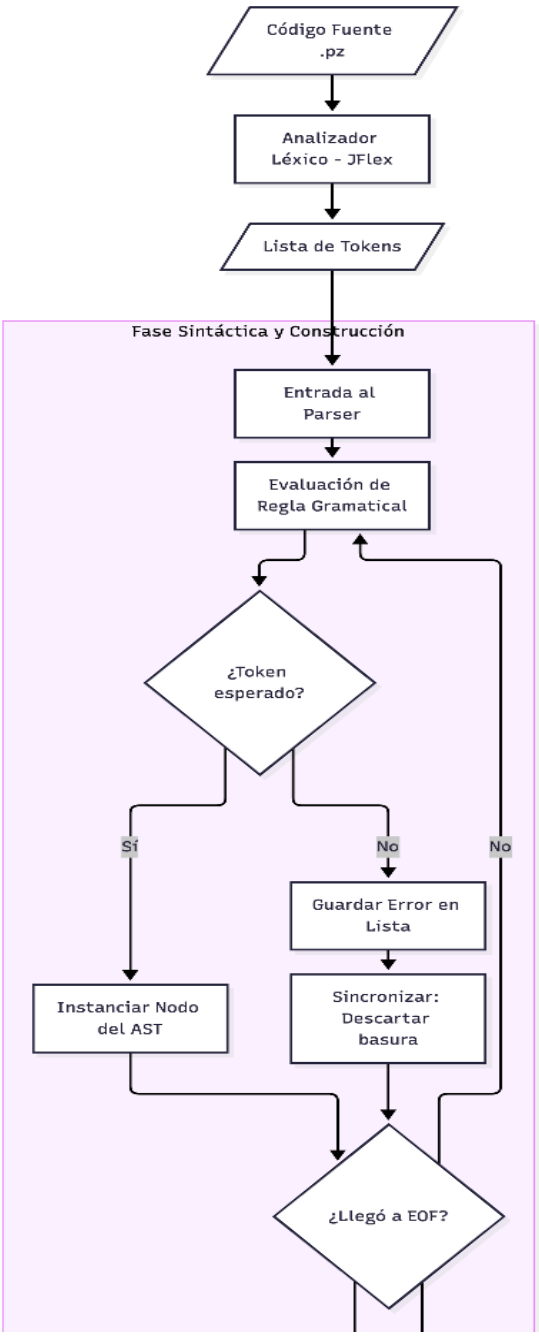
<exportar> -> EXPORTAR ID <mas_ids>

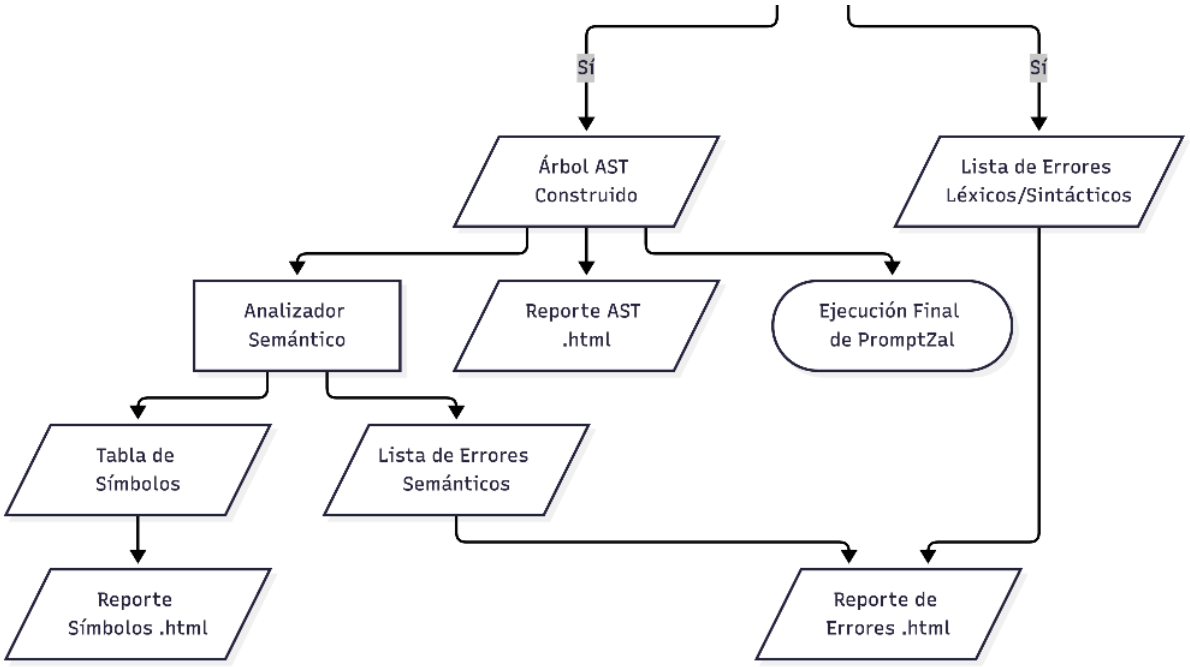
<mas_ids> -> COMA ID <mas_ids> | ε

3. Arquitectura del Sistema

3.1. Flujo General de Compilación

El ciclo de vida del código fuente en PromptZal sigue un pipeline secuencial estricto, diseñado para aislar responsabilidades. Como se observa en el siguiente diagrama, el proceso inicia cuando el entorno de desarrollo carga un archivo de código fuente (.pz).




Sí
Sí
Árbol AST Construido
Lista de Errores Léxicos/Sintácticos
Analizador Semántico
Reporte AST .html
Ejecución Final de PromptZal
Tabla de Símbolos
Lista de Errores Semánticos
Reporte Símbolos .html
Reporte de Errores .html

3.2. Análisis Léxico (Scanner)

El análisis léxico de PromptZal se encarga de convertir el código fuente (.pz) en una lista de tokens mediante la clase Lexer generada por JFlex.

3.2.1. Configuración de Directivas y Estructura

El escáner está configurado con las siguientes directivas y métodos embebidos:

- %class Lexer, %public, %type Token, %line, %unicode, %column
- Estado léxico exclusivo: %state COMENTARIO_BLOQUE
- Métodos de soporte: getErrores(), token(TokenType tipo), eof(), error(String descripcion), y el método principal tokenizar() que ejecuta un ciclo do-while con yylex() hasta alcanzar TokenType.EOF.

3.2.2. Definición de Macros

Se han establecido las siguientes expresiones regulares base:

- LETRA = [a-zA-Z_]
- DIGITO = [0-9]
- ID = {LETRA}({LETRA}|{DIGITO})*
- NUMERO = {DIGITO}+ ("." {DIGITO}+)?
- CADENA = \"[^\"\r\n]*\"
- CADENASINCERRAR = \"[^\"\r\n]*
- ESPACIO = [\t\r\n\f]+
- COMENTARIOLINEA = "//" [^\r\n]*

```
81   LETRA = [a-zA-Z_]
82
83   DIGITO = [0-9]
84
85   ID = {LETRA}({LETRA}|{DIGITO})*
86
87   NUMERO = {DIGITO}+ ("." {DIGITO}+)?
88
89   CADENA = \"[^\"\r\n]*\"
90
91   CADENASINCERRAR = \"[^\"\r\n]*
92
93   ESPACIO = [ \t\r\n\f]+
94
95   COMENTARIOLINEA = "//" [^\r\n]*
96
```

3.2.3. Reglas de Reconocimiento y Tokens (<YYINITIAL>)

- **Espacios y Comentarios de Línea:** {ESPACIO} y {COMENTARIOLINEA} se omiten.
- **Comentarios de Bloque:** Al encontrar "/*", guarda la posición inicial (comIniFila, comIniCol) y cambia al estado COMENTARIO_BLOQUE.
- **Directivas:** Se reconocen @modelo, @rol, @formato (TokenType.DIRECTIVA). Cualquier otra directiva con @ seguida de un identificador registra el error "Directiva desconocida".
- **Palabras Reservadas:** "AGENTE", "contexto", "variable", "EJECUTAR", "EXPORTAR".
- **Comandos de IA:** "PREGUNTAR", "GENERAR", "RESUMIR", "ANALIZAR", "TRADUCIR", "CLASIFICAR", "EXTRAER" (TokenType.COMANDO_IA).
- **Cargar y Conectores:** "CARGAR" (TokenType.CARGAR) y conectores "SOBRE", "DESDE", "EN", "COMO" (TokenType.CONECTOR).
- **Operadores y Delimitadores:** "→", "=", "+", "{", "}", "(", ")", ",".
- **Literales e Identificadores:** {NUMERO}, {CADENA}, y {ID}. Si se detecta {CADENASINCERRAR}, se registra el error "Cadena sin cerrar".
- **Manejo de Errores Léxicos:** Cualquier símbolo inesperado que coincida con la regla comodín [^] genera el error "Caracter no reconocido". Al llegar al final del archivo (<<EOF>>), retorna el token fin de archivo.

```
101  <YYINITIAL> {
102
103      {ESPACIO}                          { }
104      {COMENTARIOLINEA}                  { }
105
106      "/*"                               {
107                  comIniFila = yyline + 1; comIniCol = yycolumn + 1;
108                  yybegin(COMENTARIO_BLOQUE);
109                    }
110
111      "@modelo" | "@rol" | "@formato"   { return token(TokenType.DIRECTIVA); }
112      "@" {ID}                           { error("Directiva desconocida"); }
113
114      "AGENTE"                           { return token(TokenType.AGENTE); }
115      "contexto"                         { return token(TokenType.CONTEXTO); }
116      "variable"                         { return token(TokenType.VARIABLE); }
117      "EJECUTAR"                         { return token(TokenType.EJECUTAR); }
118      "EXPORTAR"                         { return token(TokenType.EXPORTAR); }
119
120      "PREGUNTAR" | "GENERAR" | "RESUMIR" | "ANALIZAR"
121      | "TRADUCIR" | "CLASIFICAR" | "EXTRAER"
122                                         { return token(TokenType.COMANDO_IA); }
123
124      "CARGAR"                           { return token(TokenType.CARGAR); }
125
126      "SOBRE" | "DESDE" | "EN" | "COMO"  { return token(TokenType.CONECTOR); }
127
128      "->"                               { return token(TokenType.FLECHA); }
129      "="                                { return token(TokenType.IGUAL); }
130      "+"                                { return token(TokenType.MAS); }
131      "{"                                { return token(TokenType.LLAVE_A); }
132      "}"                                { return token(TokenType.LLAVE_C); }
133      "("                                { return token(TokenType.PAR_A); }
134      ")"                                { return token(TokenType.PAR_C); }
135      ","                                { return token(TokenType.COMA); }
136
137      {NUMERO}                             { return token(TokenType.NUMERO); }
138      {CADENA}                             { return token(TokenType.CADENA); }
139      {CADENASINCERRAR}                    { error("Cadena sin cerrar"); }
140      {ID}                                 { return token(TokenType.ID); }
141
```

3.2.4. Control en Comentarios de Bloque (<COMENTARIO_BLOQUE>)

- Al encontrar "*/", regresa al estado inicial YYINITIAL.
- Los caracteres internos ([^]) son ignorados.
- Si el archivo finaliza (<<EOF>>) sin cerrar el comentario, registra el error "Comentario de bloque sin cerrar" utilizando la fila y columna inicial almacenadas, regresa a YYINITIAL y emite el fin de archivo.



```
<COMENTARIO_BLOQUE> {
    "*/"                            { yybegin(YYINITIAL); }

    [^]                             { }

    <<EOF>>                         { errores.add(new ErrorLexico("/*", "Comentario de bloque sin cerrar", comIniFila, c
                                      yybegin(YYINITIAL);
                                      return eof();
                }
}
```

3.2.5. Clases de Retorno y Registro: Token y ErrorLexico

Para estructurar el flujo de información resultante del análisis, el escáner empaqueta los resultados en dos entidades principales:

- **Clase Token:** Representa cada componente léxico reconocido en el código fuente. Contiene los atributos privados `tipo` (de tipo `TokenType`), `lexema` (la cadena de texto exacta capturada), `fila` y `columna`. Cuenta con un constructor parametrizado, métodos getters de acceso (`getTipo()`, `getLexema()`, `getFila()`, `getColumna()`) y una sobreescritura del método `toString()` que devuelve una representación formateada (ej. `TIPO 'lexema' (fila:columna)`).
- **Clase ErrorLexico:** Guarda los errores lexicos encontradas durante el escaneo. Almacena el `lexema` infractor, una `descripcion` textual del error, así como la `fila` y `columna` donde se originó. Incluye sus respectivos métodos getters y una sobreescritura de `toString()` estructurada para facilitar la depuración y la generación posterior del reporte de errores HTML (ej. `Error léxico (fila:columna): descripcion -> 'lexema'`).

```java
package registro;

import registro.types.TokenType;

/**
 *
 * @author dar333n
 */
public class Token {
    private TokenType tipo;
    private String lexema;
    private int fila;
    private int columna;

    public Token(TokenType tipo, String lexema, int fila, int columna) {
        this.tipo = tipo;
        this.lexema = lexema;
        this.fila = fila;
        this.columna = columna;
    }

    public TokenType getTipo() {
        return tipo;
    }

    public String getLexema() {
        return lexema;
    }

    public int getFila() {
        return fila;
    }

    public int getColumna() {
        return columna;
    }

    @Override
    public String toString(){
        return tipo + " '" + lexema + "' (" + fila + ":" + columna + ")";
    }
}
```

```java
public class ErrorLexico {
    private final String lexema;
    private final String descripcion;
    private final int fila;
    private final int columna;

    public ErrorLexico(String lexema, String descripcion, int fila, int columna) {
        this.lexema = lexema;
        this.descripcion = descripcion;
        this.fila = fila;
        this.columna = columna;
    }

    public String getLexema() {
        return lexema;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public int getFila() {
        return fila;
    }

    public int getColumna() {
        return columna;
    }

    @Override
    public String toString() {
        return "Error léxico (" + fila + ":" + columna + "): " + descripcion + " -> '" + lexema + "'";
    }
}
```

3.3. Árbol de Sintaxis Abstracta (AST)

El Árbol de Sintaxis Abstracta (AST) representa la estructura jerárquica y semántica del código fuente de PromptZal. Está implementado mediante un diseño orientado a objetos compuesto por nodos especializados que extienden de una clase base común:

3.3.1. Clases y Jerarquía de Nodos

- **Nodo:** Clase base abstracta que define la interfaz común para la representación y recorrido del árbol sintáctico.
- **Programa:** Nodo raíz que contiene la lista global de directivas, definiciones de agentes, sentencias de ejecución e instrucciones de exportación.
- **Agente:** Representa la declaración de un agente, almacenando su identificador, contexto, variables locales y bloques de comandos.
- **Variable:** Modela la declaración y asignación de variables locales o globales dentro de la estructura del programa.
- **Comando:** Nodo encargado de encapsular las instrucciones de operaciones de IA, incluyendo la expresión de entrada y el destino de asignación.
- **Conector:** Almacena la relación y conectores opcionales (SOBRE, DESDE, EN, COMO) asociados a una instrucción de comando.
- **Expresion:** Representa la evaluación de términos y operaciones de concatenación binaria dentro del código fuente.
- **Termino:** Nodo atómico que contiene valores literales (cadenas, números), identificadores de variables o llamadas a la función CARGAR.

- **Directiva:** Encapsula las configuraciones globales como @modelo, @rol y @formato asociadas a la ejecución del sistema.
- **Referencia:** Modela el acceso y uso de identificadores existentes dentro del flujo de instrucciones.



3.4. Analizador Sintáctico (Parser)

El analizador sintáctico procesa la secuencia de tokens entregada por el escáner JFlex para validar la estructura gramatical del lenguaje PromptZal mediante la técnica de descenso recursivo escrita a mano, construyendo simultáneamente el Árbol de Sintaxis Abstracta (AST).

```java
import ast.Agente;
import ast.Comando;
import ast.Conector;
import ast.Directiva;
import ast.Expresion;
import ast.Programa;
import ast.Referencia;
import ast.Termino;
import ast.Variable;
import ast.enums.TipoTermino;
import expections.ErrorPanico;
import java.util.ArrayList;
import java.util.List;
import registro.ErrorSintactico;
import registro.Token;
import registro.types.TokenType;

/**
 *
 * @author dar333n
 */
public class Parser {


    private List<Token> tokens;
    private int pos = 0;
    private List<ErrorSintactico> errores = new ArrayList<>();

    private boolean modoPanico = false;

    private boolean dentroAgente = false;

    public Parser(List<Token> tokens) {
        this.tokens = tokens; // el último token siempre es EOF
    }

    public List<ErrorSintactico> getErrores() {
        return errores;
    }

    /* INICIO: analiza todo el programa y devuelve la raíz del AST */
    public Programa parsear() {
        return programa();
    }
```

3.4.1. Mecanismo de Recuperación de Errores (Modo Pánico y Sincronización)

Para evitar la interrupción prematura del análisis ante errores sintácticos:

- **Excepción `ErrorPanico`:** Interrumpe de manera limpia la ejecución de la regla gramatical actual sin abortar el análisis global.
- **Método `fallar(String esperado)`:** Registra formalmente la incidencia en la lista de errores (`ErrorSintactico`) almacenando el token encontrado (o `EOF`) junto con lo esperado y sus coordenadas de línea y columna, activando el indicador `modoPanico`.
- **Sincronización (`sincronizar()` y `sincronizarDirectiva()`):** Descartan tokens secuencialmente mediante un bucle hasta alcanzar un punto de anclaje seguro (como `AGENTE`, `LLAVE_C`, `COMANDO_IA`, `VARIABLE`, `EJECUTAR`, `EXPORTAR` o directivas), permitiendo reanudar la validación gramatical.

```java
/* Registra el error (token encontrado, esperado, fila y columna) y devuelve la excepción */
private ErrorPanico fallar(String esperado) {
    if (!modoPanico) {
        Token t = actual();
        String encontrado;
        if (t.getTipo() == TokenType.EOF) {
            encontrado = "EOF (fin de archivo)";
        } else {
            encontrado = t.getTipo().name() + " '" + t.getLexema() + "'";
        }
        errores.add(new ErrorSintactico(encontrado, esperado, t.getFila(), t.getColumna()));
    }
    modoPanico = true;
    return new ErrorPanico();
}

/*  verifica si el token actual es de sincronización? Marca dónde empieza o termina algo */
private boolean esSincronizacion(boolean conDirectiva) {
    TokenType t = actual().getTipo();
    if (t == TokenType.EOF) {
        return true;
    }
    if (t == TokenType.LLAVE_C) {
        return dentroAgente; // una } suelta fuera de un agente se descarta
    }
    if (conDirectiva && t == TokenType.DIRECTIVA) {
        return true;
    }
    return t == TokenType.AGENTE
            || t == TokenType.COMANDO_IA
            || t == TokenType.VARIABLE
            || t == TokenType.EJECUTAR
            || t == TokenType.EXPORTAR
            || t == TokenType.CONTEXTO;
}

/* Modo pánico: descarta tokens hasta llegar a uno de sincronización (sin consumirlo) */
private void sincronizar() {
    while (!esSincronizacion(false)) {
        avanzar();
    }
}
```

3.4.2. Gestión de Apuntadores y Métodos de la Clase Parser

Cada regla de la gramática libre de contexto cuenta con un método recursivo dedicado que retorna nodos del AST:

- **`programa()`:** Raíz del análisis que secuencia las directivas, agentes, ejecuciones e instrucciones de exportación, validando el token de cierre `EOF`.

- **`agente()` y Contextos:** Gestiona la apertura de bloques mediante `AGENTE`, el nombre (`ID`), delimitación por llaves (`LLAVE_A` / `LLAVE_C`), el control del indicador `dentroAgente`, y la invocación de subreglas para contexto, variables y comandos.
- **Comandos de IA y Conectores (`comando()`, `conectores()`):** Analiza las instrucciones operativas, procesando su argumento principal en forma de expresión, conectores opcionales (`SOBRE`, `DESDE`, `EN`, `COMO`), el operador de asignación de destino `->` (`FLECHA`), y el identificador resultante.
- **Expresiones y Términos (`expresion()`, `termino()`, `cargar()`):** Desglosa operaciones de concatenación unidas por operadores +, evaluando términos atómicos (cadenas, identificadores, números con unidades opcionales) y la función del sistema `CARGAR`.
- **Control Global (`ejecuciones()` y `exportar()`):** Valida las sentencias de ejecución (`EJECUTAR ID`) y el bloque de exportación final (`EXPORTAR ID` con soporte para múltiples identificadores separados por comas mediante `masIds()`).

```java
// ================================================================
// Gramática: un método por cada no terminal
// ================================================================

// <programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF
private Programa programa() {
    List<Directiva> directivas = directivas();
    List<Agente> agentes = agentes();
    List<Referencia> ejecuciones = ejecuciones();
    List<Referencia> exportaciones = exportar();
    try {
        esperar(TokenType.EOF); // si sobran tokens se reporta el error y se ignora el resto
    } catch (ErrorPanico e) {
        // fin del análisis
    }
    return new Programa(directivas, agentes, ejecuciones, exportaciones, 1, 1);
}

// <directivas> -> DIRECTIVA CADENA <directivas> | ε
private List<Directiva> directivas() {
    List<Directiva> lista = new ArrayList<>();
    if (es(TokenType.DIRECTIVA)) {
        Directiva d = directiva();
        if (d != null) {
            lista.add(d);
        }
        lista.addAll(directivas());
    }
    // ε: no hace nada, devuelve la lista vacía
    return lista;
}

/* DIRECTIVA CADENA (una sola directiva, con su recuperación de errores) */
private Directiva directiva() {
    try {
        Token d = esperar(TokenType.DIRECTIVA);
        Token valor = esperar(TokenType.CADENA);
        return new Directiva(d.getLexema(), sinComillas(valor.getLexema()), d.getFila(), d.getColumna());
    } catch (ErrorPanico e) {
        sincronizarDirectiva();
        return null;
    }
}
```

3.5. Análisis Semántico y Tabla de Símbolos

La fase semántica dota de significado y coherencia lógica al código estructuralmente correcto representado en el AST. Se encarga de validar reglas de visibilidad, ámbitos y existencia de entidades mediante una estructura de control centralizada.

3.5.1. Arquitectura de la Tabla de Símbolos y Clases de Registro

- **Clase `Simbolo` y `TipoSimbolo`:** Modela cada entidad registrada en el programa. Almacena el `nombre` del identificador, su `tipo` (clasificado mediante el enumerador `TipoSimbolo` en `AGENTE`, `VARIABLE_DECLARADA`, o `RESULTADO_COMANDO`), el `ambito` (el agente contenedor o "todo el programa"), y la `linea` de declaración.
- **Clase `TablaSimbolos`:** Administra la lista central de símbolos (`simbolos`). Ofrece métodos especializados para declarar y verificar existencia o duplicados tanto de agentes (`declararAgente()`, `existeAgente()`, `buscarAgente()`) como de variables acotadas a su ámbito local (`declararVariable()`, `existeVariable()`, `buscarVariable()`).

```java
public class TablaSimbolos {

    private List<Simbolo> simbolos = new ArrayList<>();

    public List<Simbolo> getSimbolos() {
        return simbolos;
    }

    /*
    METODOS PARA LLENAR LA TABLA
    */

    // AGREGAR AGENTE
    public boolean declararAgente(String nombre, int linea) {
        if (existeAgente(nombre)) {
            return false;
        }
        simbolos.add(new Simbolo(nombre, TipoSimbolo.AGENTE, "todo el programa", linea));
        return true;
    }

    public boolean existeAgente(String nombre) {
        return buscarAgente(nombre) != null;
    }

    public Simbolo buscarAgente(String nombre) {
        for (Simbolo s : simbolos) {
            if (s.getTipo() == TipoSimbolo.AGENTE && s.getNombre().equals(nombre)) {
                return s;
            }
        }
        return null;
    }


    // AGREGAR VARIABLE
    public boolean declararVariable(String agente, String nombre, TipoSimbolo tipo, int linea) {
        if (existeVariable(agente, nombre)) {
            return false;
        }
        simbolos.add(new Simbolo(nombre, tipo, agente, linea));
```

```java
/**
 *
 * @author dar333n
 */
public class Simbolo {

    private String nombre;
    private TipoSimbolo tipo;
    private String ambito;
    private int linea;

    public Simbolo(String nombre, TipoSimbolo tipo, String ambito, int linea) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.ambito = ambito;
        this.linea = linea;
    }

    public String getNombre() {
        return nombre;
    }

    public TipoSimbolo getTipo() {
        return tipo;
    }

    public String getAmbito() {
        return ambito;
    }

    public int getLinea() {
        return linea;
    }


}
```

3.5.2. Proceso de Análisis y Validación (AnalizadorSemantico)

El método principal `analizar(Programa programa)` coordina el recorrido del AST en etapas secuenciales:

```java
public class AnalizadorSemantico {

    private TablaSimbolos tabla = new TablaSimbolos();
    private List<ErrorSemantico> errores = new ArrayList<>();

    public TablaSimbolos getTabla() {
        return tabla;
    }

    public List<ErrorSemantico> getErrores() {
        return errores;
    }

    /*
    INICIO: Analizador semantico
    */
    public void analizar(Programa programa) {

        // ANALIZA LOS AGENTES Y SUS VARIABLES
        for (Agente agente : programa.getAgentes()) {
            analizarAgente(agente);
        }

        // ANALIZAR LAS EJECUCIONES
        List<String> ejecutados = validarEjecuciones(programa);


        // ANALIZAR LOS ID DEL EXPORT
        validarExportaciones(programa, ejecutados);
    }
```

- **1. Validación de Agentes:** Itera sobre la lista de agentes para verificar su unicidad mediante `tabla.declararAgente()`. Si un agente se reencuentra duplicado, registra inmediatamente un `ErrorSemantico` con la línea y columna originales.

```
private void analizarAgente(Agente agente){

    // Verificar que el agente no este repetido con el metodo de tabla
    boolean declarado = tabla.declararAgente(agente.getNombre(), agente.getLinea());
    if (!declarado){

        Simbolo primero = tabla.buscarAgente(agente.getNombre());
        error("El agente "+agente.getNombre()+" ya existe en la linea "+primero.getLinea(), agente.getLinea(), agente.g
        return;
    }

    for (Variable variable : agente.getVariables()){ // Obtenemos las variables del agente que estamos abalizando

        validarExpresion(variable.getValor(), agente.getNombre(), false);

        tabla.declararVariable(agente.getNombre(), variable.getNombre(),
                TipoSimbolo.VARIABLE_DECLARADA, variable.getLinea());
    }


    for (Comando comando : agente.getComandos()){ // Obtenemos los comandos del agente

        boolean esGenerar = comando.getTipoComando().equals("GENERAR");
        validarExpresion(comando.getArgumento(), agente.getNombre(), esGenerar);

        for (Conector conector : comando.getConectores()){
            validarExpresion(conector.getArgumento(), agente.getNombre(), false);
        }

        // lo que va despues de -> tambien declara una variable
        Referencia destino = comando.getDestino();
        tabla.declararVariable(agente.getNombre(), destino.getNombre(), TipoSimbolo.RESULTADO_COMANDO,
                destino.getLinea());
    }
}
```

- **2. Procesamiento de Variables y Comandos Locales:** Por cada agente analizado:
    - Se validan las expresiones de declaración de variables con `validarExpresion()`, registrándolas en el ámbito local del agente (`TipoSimbolo.VARIABLE_DECLARADA`).
    - Se procesan los comandos de IA, validando sus argumentos y conectores, y registrando formalmente las variables destino (después de la flecha `->`) como `TipoSimbolo.RESULTADO_COMANDO`.
- **3. Validación de Expresiones (`validarExpresion`):** Recorre los términos de una expresión y comprueba que cualquier identificador (`ID`) utilizado esté previamente declarado dentro del agente actual. Se contempla la excepción específica para

```
// VAlidar los ID
private void validarExpresion(Expresion expresion, String nombreAgen, boolean primerIdEsTipo) {

    List<Termino> terminos = expresion.getTerminos();
    for (int i = 0; i < terminos.size(); i++){
        Termino t = terminos.get(i);

        // Verificar si es un ID
        if (t.getTipo() != TipoTermino.ID){
            continue;
        }

        // Verfica que no sea una variable
        if (primerIdEsTipo && i == 0){
            continue;
        }

        // Verifica que la variabe no exista ya
        if (!tabla.existeVariable(nombreAgen, t.getValor())){
            error("La variable " + t.getValor() + " no está declarada en el agente "+nombreAgen,
                    t.getLinea(), t.getColumna());
        }
    }
}
```

comandos como `GENERAR`, donde el primer identificador indica el tipo de contenido y no una variable declarada.

- **4. Validación de Sentencias de Ejecución (`validarEjecuciones`):** Comprueba que cada identificador citado en los bloques `EJECUTAR` corresponda a un agente real existente en la tabla de símbolos. Si no existe, se emite un error semántico.

```
// Validar las ejecuciones
private List<String> validarEjecuciones(Programa programa) {
    List<String> ejecutados = new ArrayList<>();
    for (Referencia ejecucion : programa.getEjecuciones()) {
        if (tabla.existeAgente(ejecucion.getNombre())) {
            ejecutados.add(ejecucion.getNombre());
        } else {
            error("El agente " + ejecucion.getNombre() +" no existe",
                    ejecucion.getLinea(), ejecucion.getColumna());
        }
    }
    return ejecutados;
}
```

- **5. Comprobación de Exportaciones (`validarExportaciones`):** Garantiza que cada variable indicada en el bloque `EXPORTAR` pertenezca efectivamente a alguno de los agentes que fueron ejecutados exitosamente, previniendo fugas de datos o exportaciones de elementos nulos.

```
// Validaer las exportaciones
private void validarExportaciones(Programa programa, List<String> ejecutados) {
    for (Referencia exportada : programa.getExportaciones()) {
        boolean existe = false;
        for (String agente : ejecutados) {
            if (tabla.existeVariable(agente, exportada.getNombre())) {
                existe = true;
            }
        }

        if (!existe) {
            error("La variable " + exportada.getNombre() + " no existe en ninguno de los agentes ejecutados",
                    exportada.getLinea(), exportada.getColumna());
        }
    }
}
```

3.5.3. Sistema de Registro de Errores Semánticos

Utiliza la clase `ErrorSemantico` para almacenar incidencias (descripción textual, fila y columna), permitiendo recopilar múltiples fallas semánticas por corrida sin interrumpir drásticamente la ejecución del compilador.

```java
//Registrar nuevo error
private void error(String descripcion, int linea, int columna) {
    errores.add(new ErrorSemantico(descripcion, linea, columna));
}
```

3.6. Generación de Código (Compilador)

El compilador procesa el código secuencialmente a través de sus fases léxica, sintáctica y semántica para luego generar el archivo traducido

3.6.1. Clases Principales(Compilador y DatosCompilacion)

- **Compilador:** Su método principal `compilar(String codigo)`. Instancia secuencialmente el `Lexer` para obtener la lista de tokens, el `Parser` para construir el Árbol de Sintaxis Abstracta (AST), y finalmente el `AnalizadorSemantico` para validar los ámbitos.

```java
 * @author dar333n
 */
public class Compilador {

    public DatosCompilacion compilar(String codigo) {
        // lexer
        Lexer lexer = new Lexer(new StringReader(codigo));
        List<Token> tokens = new ArrayList<>();
        try {
            tokens = lexer.tokenizar();
        } catch (IOException e) {
            System.out.println("Error leyendo el codigo fuente: " + e.getMessage());
        }
        List<ErrorLexico> erroresLexicos = lexer.getErrores();

        // Parser
        Parser parser = new Parser(tokens);
        Programa programa = parser.parsear();
        List<ErrorSintactico> erroresSintacticos = parser.getErrores();

        // Analizador semantico y promptZAl
        TablaSimbolos tabla = new TablaSimbolos();
        List<ErrorSemantico> erroresSemanticos = new ArrayList<>();
        String promptzal = null;

        if (erroresLexicos.isEmpty() && erroresSintacticos.isEmpty()) {
            AnalizadorSemantico semantico = new AnalizadorSemantico();
            semantico.analizar(programa);
            tabla = semantico.getTabla();
            erroresSemanticos = semantico.getErrores();

            promptzal = PromptZal.generar(programa);
        }

        return new DatosCompilacion(tokens, erroresLexicos, erroresSintacticos,
                erroresSemanticos, programa, tabla, promptzal);
    }
}
```

- **DatosCompilacion:** Modela un contenedor de transporte (DAO) que empaqueta todos los resultados del proceso (tokens, AST, tabla de símbolos, código PromptZal resultante y listas de errores). Esto permite a la Interfaz Gráfica (GUI) acceder a la información para renderizar los reportes sin acoplarse directamente a la lógica de análisis.

```java
public class DatosCompilacion {


    private List<Token> tokens;
    private List<ErrorLexico> erroresLexicos;
    private List<ErrorSintactico> erroresSintacticos;
    private List<ErrorSemantico> erroresSemanticos;
    private Programa programa;
    private TablaSimbolos tabla;
    private String promptzal;

    public DatosCompilacion(List<Token> tokens, List<ErrorLexico> erroresLexicos, List<ErrorSintactico> erroresSintacticos, |
        this.tokens = tokens;
        this.erroresLexicos = erroresLexicos;
        this.erroresSintacticos = erroresSintacticos;
        this.erroresSemanticos = erroresSemanticos;
        this.programa = programa;
        this.tabla = tabla;
        this.promptzal = promptzal;
    }

    public List<Token> getTokens() {
        return tokens;
    }

    public List<ErrorLexico> getErroresLexicos() {
        return erroresLexicos;
    }

    public List<ErrorSintactico> getErroresSintacticos() {
        return erroresSintacticos;
    }

    public List<ErrorSemantico> getErroresSemanticos() {
        return erroresSemanticos;
    }

    public Programa getPrograma() {
        return programa;
    }
```

3.6.2. Clase PromptZal

La clase `PromptZal` se encarga de recorrer el AST validado y traducir las instrucciones del lenguaje PromptZal a lenguaje natural (prompts de sistema) mediante el método `generar(Programa programa)`:

- **Gestión de Directivas:** Captura las configuraciones globales (`@modelo`, `@rol`, `@formato`) para establecer el encabezado del prompt generado.
- **Ensamblaje de Agentes (Filtro de Ejecución):** Itera sobre las instrucciones `EJECUTAR`. Extrae únicamente los agentes que han sido invocados explícitamente, evitando traducciones redundantes y descartando código inalcanzable. Se documenta su contexto, variables disponibles e instrucciones.
- **Resolución de Variables y Términos (origenVariable y textoExpresion):** Transforma la evaluación de términos en descripciones explícitas (ej. mapea `CARGAR` a *"contenido del archivo..."* o `CADENA` a *"el texto..."*).

- **Traducción de Comandos y Conectores (instruccion):** Traduce cada operación de IA según su tipo semántico utilizando plantillas de texto específicas. Para cada comando, el código maneja **dos variantes** dependiendo de si el usuario incluyó o no un conector opcional (`SOBRE`, `DESDE`, `EN`):
    - **`PREGUNTAR`:** "Responde: [x]". Con conector `SOBRE`: "Basándote en [sobre], responde: [x]".
    - **`GENERAR`:** "Genera [x]". Con conector `DESDE`: "Genera [x] a partir de: [desde]".
    - **`RESUMIR`:** "Resume [x]". Con conector `EN`: "Resume [x] en un máximo de [en]".
    - **`ANALIZAR`:** "Analiza [x]". Con conector `SOBRE`: "Analiza [x] considerando: [sobre]".
    - **`TRADUCIR`:** "Traduce [x]". Con conector `EN`: "Traduce [x] al idioma [en]".
    - **`CLASIFICAR`:** "Clasifica [x]". Con conector `EN`: "Clasifica [x] en las categorías: [en]".
    - **`EXTRAER`:** "Extrae [x]". Con conector `DESDE`: "Extrae [x] a partir de [desde]".
    - Adicionalmente, al final de cada frase se concatena de forma dinámica la cláusula de guardado ("Guarda el resultado como [destino]") y, si se evalúa el conector `COMO`, se adjunta la instrucción de formato ("Entrega el resultado en formato [como]").
- **Exportaciones:** Concluye la traducción generando el bloque de "Resultado solicitado" basado en los identificadores declarados en el bloque `EXPORTAR`.

```java
public class PromptZal {

    public static String generar(Programa programa) {
        String texto = "# Prompt generado con PromptZal\n";

        // Buscar las directivas agregadas
        String modelo = buscarDirectiva(programa, "@modelo");
        String rol = buscarDirectiva(programa, "@rol");
        String formato = buscarDirectiva(programa, "@formato");
        if (modelo != null) {
            texto = texto + "Modelo destino: " + modelo + "\n";
        }
        if (rol != null) {
            texto = texto + "Rol general: " + rol + "\n";
        }
        if (formato != null) {
            texto = texto + "Formato de respuesta: " + formato + "\n";
        }

        // Procesar cada agente que si se ejecuto por separado en el for
        List<String> yaIncluidos = new ArrayList<>();
        for (Referencia ejecucion : programa.getEjecuciones()){
            Agente agente = buscarAgente(programa, ejecucion.getNombre());

            // Verificar de agentes
            if (agente == null) {
                continue; // el agente no se ejecuto, no se toma encuenta y ya debe salir marcado como error
            }
            if (yaIncluidos.contains(agente.getNombre())) {
                continue; // si se ejecuta dos veces, se incluye una sola vez
            }

            // Si pasa la veriifcacion se agrega, a la lista y al texto
            yaIncluidos.add(agente.getNombre());
            texto = texto + generarAgente(agente);
        }

        //
        texto = texto + "\n## Resultado solicitado\n";
        List<String> yaPedidos = new ArrayList<>();
        for (Referencia exportada : programa.getExportaciones()) {
            if (!yaPedidos.contains(exportada.getNombre())) {
                yaPedidos.add(exportada.getNombre());
                texto = texto + "- " + exportada.getNombre() + "\n";
            }
```

3.7. Sistema de Manejo y Reporte de Errores

El compilador está diseñado para ser tolerante a fallos. En lugar de detener su ejecución ante la primera equivocación, el sistema captura, clasifica y almacena las incidencias encontradas en cada fase para presentarlas al usuario mediante reportes.

3.7.1. Clasificación y Detección de Errores

- **Errores Léxicos:** Detectados por el escáner (JFlex) al encontrar caracteres que no pertenecen al alfabeto del lenguaje (mediante la regla comodín `[^]`).
- **Errores Sintácticos:** Identificados por el `Parser` cuando la secuencia de tokens no respeta las reglas de la gramática libre de contexto.
- **Errores Semánticos:** Capturados por el `AnalizadorSemantico` al encontrar incoherencias lógicas (ej. variables no declaradas, agentes duplicados o exportaciones inválidas) al recorrer el AST.

3.7.2. Entidades de Registro (Clases de Error)

Para mantener un estándar de reporte estructurado, el sistema utiliza tres clases especializadas que almacenan la información precisa de cada fallo:

- **Clase `ErrorLexico`:** Almacena el `lexema` infractor, una `descripcion` textual del fallo y las coordenadas exactas (`fila` y `columna`). Sobrescribe `toString()` para generar mensajes del tipo: `Error léxico (fila:columna): descripcion -> 'lexema'`.
- **Clase `ErrorSintactico`:** Registra el token `encontrado` y el token `esperado`, además de sus coordenadas (`fila` y `columna`). Su salida se formatea como: `Error sintáctico (fila:columna): se esperaba X, llegó Y`.
- **Clase `ErrorSemantico`:** Almacena la `descripcion` de la regla semántica violada junto con su ubicación (`fila` y `columna`). Se reporta como: `Error semántico (fila:columna): descripcion`.

3.7.3. Recuperación de errores

La tolerancia a fallos opera de manera transversal a lo largo del proceso de compilación:

- **En la fase Léxica:** El escáner anota el carácter inválido y continúa leyendo el flujo de entrada.
- **En la fase Sintáctica:** Se emplea el **Modo Pánico** y métodos de **Sincronización** (`sincronizar()`), permitiendo al parser descartar tokens problemáticos hasta encontrar un punto de anclaje seguro y reanudar el análisis sin abortar el proceso.
- **En la fase Semántica:** El analizador almacena el error en su lista interna y prosigue con la evaluación de las siguientes expresiones e instrucciones del AST.

3.7.4. Reporte HTML

Finalmente, la clase `Compilador` recopila las tres listas de errores (léxicos, sintácticos y semánticos) y las empaqueta dentro del objeto `DatosCompilacion`. Esto permite que la Interfaz Gráfica (GUI) extraiga esta información desacoplada para renderizar de manera limpia y ordenada el **Reporte de Errores en formato HTML**, informando al programador de todas las fallas en una sola corrida o confirmando el éxito si las listas están vacías.